

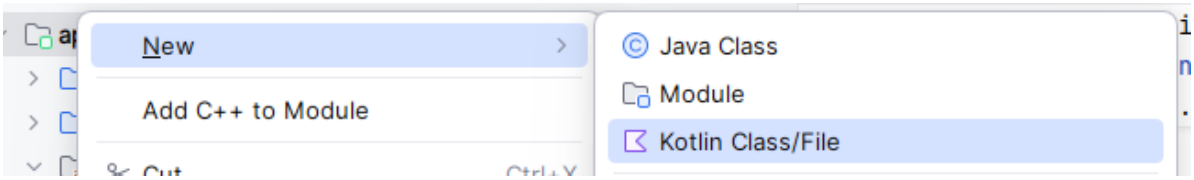
Aplicaciones Móviles

DESARROLLO DE APLICACIONES MOVILES EN ANDROID
IMPLEMENTACIÓN CRUD SQL-LITE

CRUD + SQLI

Abrir un nuevo proyecto en blanco

Se debe crear una clase que permita crear la tabla y los datos para la conexión, para este caso le llamaremos **ConexionDbHelper**



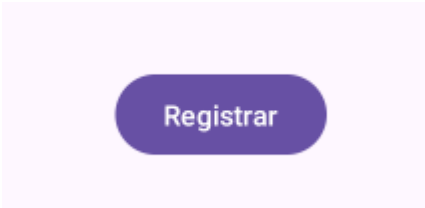

```
class ConexionDbHelper(
    context: Context
) : SQLiteOpenHelper(context, DATABASE_NAME, null, DATABASE_VERSION) {


val sql="CREATE TABLE USUARIOS (ID INTEGER PRIMARY KEY, NOMBRE TEXT, APELLIDO TEXT, EMAIL TEXT, CLAVE TEXT)";
    override fun onCreate(db: SQLiteDatabase) {
        // Aquí va tu SQL para crear tablas
        db.execSQL(sql)
    }
    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        // Aquí puedes actualizar tu esquema
        db.execSQL("DROP TABLE USUARIOS")
        onCreate(db)
    }
    companion object {
        private const val DATABASE_NAME = "CRUD"
        private const val DATABASE_VERSION = 1
    }
}
```

Crearan un Activity en Blanco con el Nombre de **Registro,** con la siguiente distribución de elementos básicos para este registro



Realizaremos en el Activity principal un botón que permita registrar, cargaremos el activity creado recientemente.



Creamos un método llamado registrar()

```
btn1=findViewById(R.id.button)

btn1.setOnClickListener()
{
    val registro = Intent(this, Registro::class.java)
    startActivity(registro)
}
```

Ahora en el Activity registro realizaremos la captura de los datos y los registraremos en nuestra base de datos.

```
lateinit var nombre: EditText
lateinit var apellido: EditText
lateinit var email: EditText
lateinit var clave: EditText
lateinit var btn_reg: Button
```

Y realizamos el casteo de los datos en las variables declaradas

```
nombre=findViewById(R.id.txtnombre)
apellido=findViewById(R.id.txtapellido)
email=findViewById(R.id.txtemail)
clave=findViewById(R.id.txtclave)
btn_reg=findViewById(R.id.btn_registro)
```

Crearemos un método que nos permita guardar los datos que llamaremos **guardar()**

Donde crearemos la Base de Datos **"APPSQLITE"** y enviaremos los datos por medio de un ContentValues

```
val helper = ConexionDbHelper(this)

fun guardar(nom: String, ape: String, mai: String, cla: String) {
    val helper = ConexionDbHelper(this)
    val db = helper.readableDatabase
    try {
        val datos = ContentValues().apply {
            put("Nombre", nom)
            put("Apellido", ape)
            put("Email", mai)
            put("Clave", cla)
        }
        db.insert("USUARIOS", null, datos)
        Toast.makeText(this, "Datos ingresados sin problemas",
Toast.LENGTH_LONG).show()

    } catch (e: Exception) {
        Toast.makeText(this, "Error: ${e.message}",
Toast.LENGTH_LONG).show()
    }
}
```

Ahora le decimos al botón cuando sea presionado que guarde los datos ingresados, con el método Creado.

```
btn_reg.setOnClickListener(View.OnClickListener {

    val nombreText = nombre.text.toString().trim()
    val apellidoText = apellido.text.toString().trim()
    val emailText = email.text.toString().trim()
    val claveText = clave.text.toString().trim()

        guardar(
            nombre.text.toString(),
            apellido.text.toString(),
```

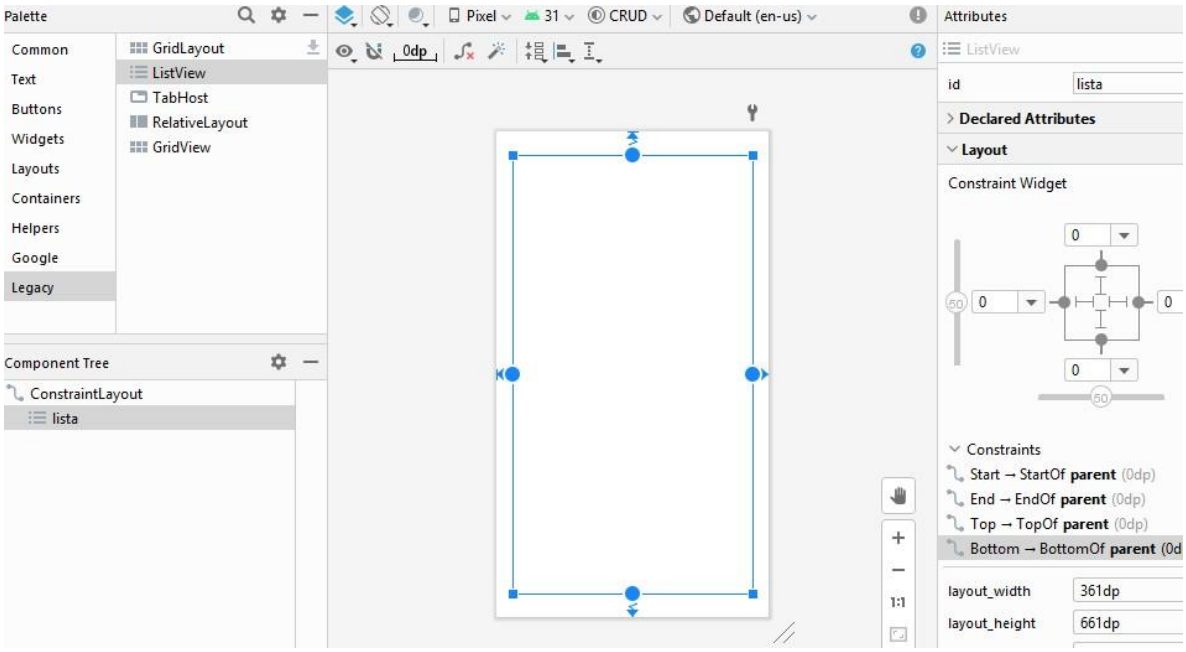
```
            email.text.toString(),
            clave.text.toString()
        )
})
```

Se realiza la Prueba y una vez ingresados los datos deber aparecer que los datos fueron ingresados sin problemas.



Ahora para poder visualizar los datos si se ingresaron o no crearemos otro activity que le llamaremos **Listado**, donde mostraremos todos los datos que se encuentran en nuestra tabla.

En el diseño agregaremos un **ListView** que le llamaremos **lista**, como se muestra en la siguiente imagen.



Crearemos dos variables que utilizaremos para realizar la muestra

```
private lateinit var listado: ListView
private lateinit var listausuario: ArrayList<String>
```

Luego crearemos un método donde Listaremos todos los usuarios de tabla como un ArrayList

```
private fun listaUsuario(): ArrayList<String> {
    val datos = ArrayList<String>()
    val helper = ConexionDbHelper(this)
    val db = helper.readableDatabase
    val sql = "SELECT * FROM USUARIOS"
    val c = db.rawQuery(sql, null)

    if (c.moveToFirst()) {
        do {
            val linea = "${c.getInt(0)} ${c.getString(1)}
${c.getString(2)}"
            datos.add(linea)
        } while (c.moveToNext())
    }

    c.close()
    db.close()

    return datos
}
```

Como ya tenemos en el método creado el listado de todos los usuarios que existen en la tabla de nuestra base de datos, pero están en formato ArrayList, deberemos pasarlo a un ListView que fue el

que se definió en el diseño de nuestro Activity, para eso crearemos un método CargarLista(), para adaptar nuestro array al ListView.

```
private fun CargarLista() {
    listausuario = listaUsuario()
    val adapter = ArrayAdapter(this, android.R.layout.simple_list_item_1,
listausuario)
    listado.adapter = adapter
}
```

Ahora Casteamos los datos y llamamos al método CargarLista()

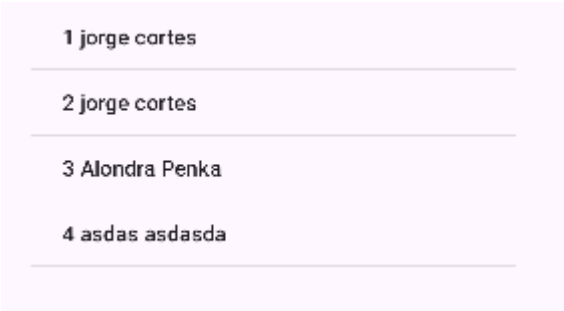
```
listado= findViewById(R.id.lista);
CargarLista()
```

Y Agregaremos las siguientes líneas en el Registro para una vez ingresado el usuario mostraremos la Lista de Usuarios

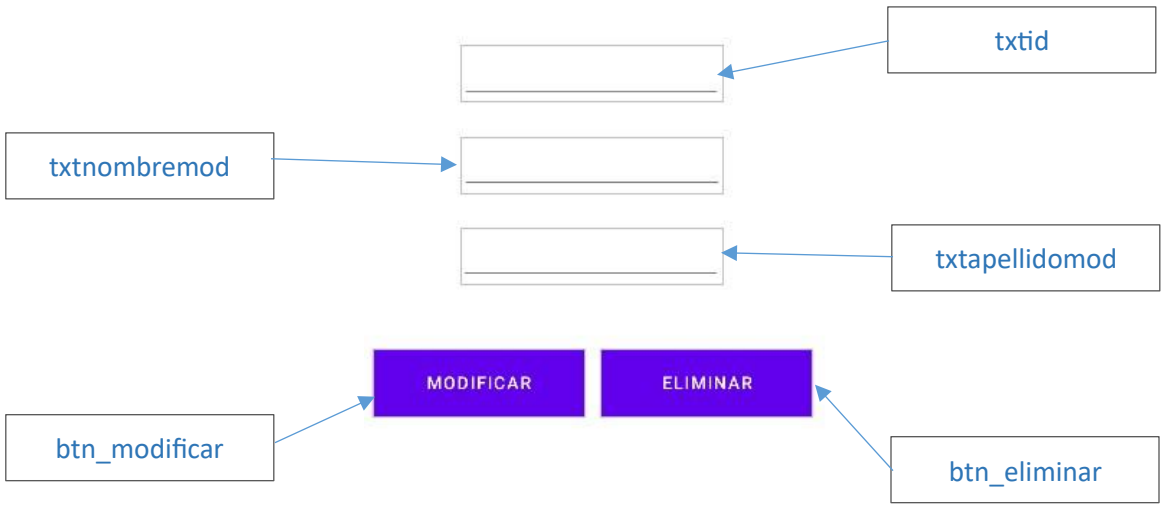
```
Toast.makeText(this, "Datos ingresados sin problemas",
Toast.LENGTH_LONG).show()

val listado = Intent(this, Listado::class.java)
startActivity(listado)
```

Realizamos la Prueba y deberíamos obtener como resultado después de ingresar los datos la muestra de los usuarios ingresados los datos.



Ahora crearemos un activity para modificar y Eliminar los datos de los clientes, la idea será hacerlo cuando mostremos la lista y seleccionemos un usuario, para eso crearemos este activity como ModificarEliminar, con el siguiente diseño.



Castearemos los objetos para poder utilizarlos.

```
lateinit var id: EditText
lateinit var nom: EditText
lateinit var ape: EditText
lateinit var mod: Button
lateinit var elim: Button


id = findViewById(R.id.txtid)
nom = findViewById(R.id.txtnombremod)
ape = findViewById(R.id.txtapellidomod)
mod = findViewById(R.id.btn_modificar)
elim = findViewById(R.id.btn_eliminar)
```

Ahora iremos al activity del **listado** e implementaremos la opción cuando seleccionemos el ítem del listview esto lo haremos dentro del Método OnCreate

```
listado.onItemClickListener =
AdapterView.OnItemClickListener { parent, view,
position, id ->
    val item = listausuario[position]
    val datos = item.split(" ")

    if (datos.size >= 3) {
        val idusu = datos[0].toIntOrNull() ?: 0
        val nombre = datos[1]
        val apellido = datos[2]


        val intent = Intent(this@Listado,
ModificarEliminar::class.java).apply {
            putExtra("Id", idusu)
```

```
            putExtra("Nombre", nombre)
            putExtra("Apellido", apellido)
        }
        startActivity(intent)
    }
}
```

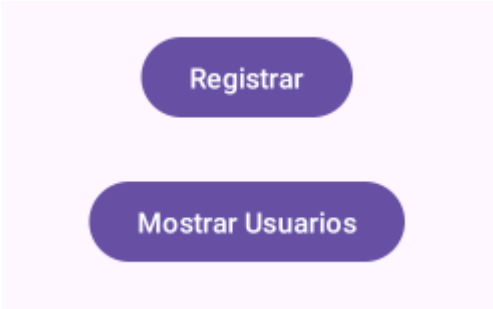
De esta manera enviaremos los datos seleccionados al Activity **ModificarEliminar** con la opción de **putExtra**

Ahora deberemos capturar los datos en el Activity **ModificarEliminar**

```
val idusu = intent.getIntExtra("Id", 0)
val nombre = intent.getStringExtra("Nombre") ?: ""
val apellido = intent.getStringExtra("Apellido") ?: ""

id.setText(idusu.toString())
nom.setText(nombre)
ape.setText(apellido)
```

Ahora para poder realizar el ciclo completo, realizaremos algunas modificaciones el Activity Principal, agregaremos un nuevo botón para mostrar a los usuarios.



Y realizaremos el llamado al listado

```
btn2.setOnClickListener()
{
    val listado = Intent(this, Listado::class.java)
    startActivity(listado)
}
```

Realizamos la Prueba y debería cargar los datos después de seleccionar un usuario de la lista.

Ahora implementaremos los métodos modificar y eliminar.

```
private fun modificar(id: Int, nombre: String,
apellido: String) {
    val helper = ConexionDbHelper(this)
    val db = helper.writableDatabase
    val sql = "UPDATE USUARIOS SET NOMBRE='$nombre',
```

```
APELLIDO='$apellido' WHERE ID=$id"
    db.execSQL(sql)
    db.close()
}

private fun eliminar(id: Int) {
    val helper = ConexionDbHelper(this)
    val db = helper.writableDatabase
    val sql = "DELETE FROM USUARIOS WHERE ID=$id"
    db.execSQL(sql)
    db.close()
}
```

y Habilitamos ahora los envió de cada botón

```
mod.setOnClickListener {
     modificar(idusu, nom.text.toString(),
ape.text.toString())
     onBackPressedDispatcher.onBackPressed()
 }


elim.setOnClickListener {
    eliminar(idusu)
    onBackPressedDispatcher.onBackPressed()
}
```

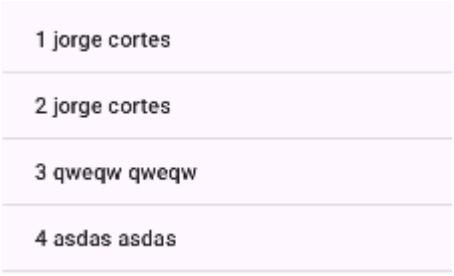
la Opción **onBackPressed();** hace que vuelva al Activity anterior, pero deberemos realizar una modificación en el listado para que refresque la lista con los datos modificados o eliminados.

```
override fun onStart() {
    super.onStart()
    CargarLista()
}
```

Y realizamos la Prueba.

Si volvemos atrás cerramos el activity en ejecución y liberamos memoria y recurso del celular, y en el método onCreate habilitamos la opción volver.

```
supportActionBar?.apply {
    setDisplayHomeAsUpEnabled(true)
    setDisplayShowHomeEnabled(true)
}
```



Probamos y tenemos el Crud 100% Funcionando

Ahora implementaremos un mensaje al momento de ingresar, en el Activiti de Ingreso creamos el siguiente método

```
private fun mostrarAlertaCamposVacios() {
    SweetAlertDialog(this,
SweetAlertDialog.WARNING_TYPE)
        .setTitleText("Campos incompletos")
        .setContentText("Por favor, complete todos los
campos antes de continuar.")
        .setConfirmText("Aceptar")
        .setConfirmClickListener { dialog ->
            dialog.dismissWithAnimation()
        }
        .show()
}
```

y antes de ingresar validamos los campos de ingreso

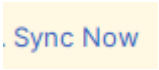
```
if (nombreText.isBlank() || apellidoText.isBlank() ||
emailText.isBlank() || claveText.isBlank()) {
    mostrarAlertaCamposVacios()
} else {
    guardar(
        nombre.text.toString(),
        apellido.text.toString(),
        email.text.toString(),
        clave.text.toString()
    )
}
```

ahora deberemos incorporar las dependencias y modificaremos el archivo **build.gradle**

```
implementation("com.github.f0ris.sweetalert:library:1.
                        6.2")
```

```
implementation("com.github.f0ris.sweetalert:library:1.6.2")
```

Y realizamos la sincronización

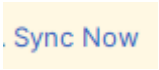


También deberemos modificar el archivo **settings.gradle**

```
maven(url = https://jitpack.io
```

```
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        maven(url = "https://jitpack.io")
    }
```

Y realizamos la sincronización



Y ahora simplemente queda probar la aplicación y debes implementar los mensajes según corresponda pata la Modificación y Eliminación, para eso debes ver el manual creado y que se encuentra en el ambiente **IMPLEMENTACIÓN DE LIBRERÍA SWEETALERT DIALOG.**